

README / Setup Guide

Overview

This application is designed to manage employee leave requests within an organization. It allows employees to submit leave requests, view their leave balances, and managers to approve or reject these requests. The core benefits include streamlined leave management, improved transparency, and efficient tracking of leave balances. The target users are employees and managers within the organization.

Key Features

- **Leave Request Submission:** Employees can submit leave requests through a user-friendly interface.
- **Leave Balance Retrieval:** Employees can view their current leave balances.
- **Leave Request Approval Workflow:** Managers can approve or reject leave requests.
- **Validation and Error Handling:** Ensures data integrity and provides meaningful error messages.

Prerequisites

- **Java 11+:** Ensure Java 11 or higher is installed.
- **Node.js 16+:** Required for frontend dependencies.
- **PostgreSQL 13+:** For the database.
- **Git:** For cloning the repository.
- **Docker:** Optional, for running the database in a container.

Project Structure

The repository is structured as follows:

- **backend/:** Contains the Spring Boot application with REST controllers, service classes, repositories, and security configurations.
- **frontend/:** Contains the React application with components, services, and API calls.
- **shared/:** Contains shared utilities if any.
- **docs/:** Contains documentation and API specifications.

Installation & Setup

Backend Setup

1. Clone the repository: `git clone <url>`
2. Navigate to the project: `cd employee-leave-app`
3. Create virtual environment: `python -m venv venv`
4. Activate: `source venv/bin/activate` (Linux/Mac) or `venv\Scripts\activate` (Windows)
5. Install dependencies: `pip install -r backend/requirements.txt`
6. Configure PostgreSQL database by updating `backend/src/main/resources/application.properties` with your database credentials.
7. Set up Spring Security by configuring `SecurityConfig.java` in `backend/src/main/java/com/example/leave/security/`.

Frontend Setup

1. Navigate to frontend: `cd frontend`
2. Install dependencies: `npm install`
3. Build/prepare assets: `npm run build`

Configuration

Required Environment Variables

- `SPRING_DATASOURCE_URL`: Controls the database connection URL, format: `jdbc:postgresql://localhost:5432/leavedb`, example value: `jdbc:postgresql://localhost:5432/leavedb`.
- `SPRING_DATASOURCE_USERNAME`: Controls the database username, format: `string`, example value: `postgres`.
- `SPRING_DATASOURCE_PASSWORD`: Controls the database password, format: `string`, example value: `password`.

Optional Environment Variables

- `JWT_SECRET` (default: `secret`): Controls the JWT secret key used for authentication.

Running the Application

Starting the Backend

1. Ensure virtual environment is activated.
2. Set environment variables: `export SPRING_DATASOURCE_URL=jdbc:postgresql://localhost:5432/leavedb`
3. Start server: `cd backend && mvn spring-boot:run`

4. Verify: Backend is running on `http://localhost:8080`

Starting the Frontend

1. Navigate to frontend directory: `cd frontend`
2. Start dev server: `npm run dev`
3. Open browser: `http://localhost:3000`
4. Verify: UI loads and connects to backend

Verification Checklist

- ☐ Backend health check responds at `GET /api/health` with 200 OK
- ☐ Frontend loads without console errors
- ☐ Basic workflow completes end-to-end (e.g., employee submits a leave request and manager approves it)
- ☐ API endpoints respond with expected data format

Troubleshooting

Backend Issues

- **Port already in use:** Change `PORT` env var or kill process on port 8080.
- **Module not found:** Run `pip install -r backend/requirements.txt` again.
- **Database connection error:** Verify connection string in `application.properties`, check service is running.
- **Authentication error:** Verify JWT secret in `application.properties` and API keys/tokens in `.env` are correct and not expired.

Frontend Issues

- **Blank page or won't load:** Check browser console for errors, ensure backend is running.
- **API connection error:** Verify backend URL in config, check CORS settings.
- **Module resolution error:** Delete `node_modules` and run `npm install` again.
- **Build fails:** Ensure Node.js version matches requirements, clear cache: `npm cache clean --force`.

Common Workflows

Employee Submits Leave Request

1. Navigate to leave request form.

2. Fill in leave details: employee ID, leave type, start date, end date.
3. Submit the form.
4. Verify: Leave request is created with status "PENDING".

Manager Approves Leave Request

1. Navigate to leave request details.
2. Click "Approve" button.
3. Verify: Leave request status changes to "APPROVED".

Deployment Notes

- Ensure environment variables are set correctly in the production environment.
- Use a secrets management tool for sensitive information like database credentials and JWT secret.
- Consider using a reverse proxy like Nginx for production deployments.

Getting Help

- Check logs: Backend logs in console, frontend logs in browser DevTools.
- Review error message and stack trace for clues.
- Check environment variables are set correctly.
- Refer to the API Documentation and User Guide for more details.